Dimensionality Reduction

transformation of data from a high-dimensional space into a low-dimensional space
so that the low-dimensional representation retains some meaningful properties of the original data,
ideally close to its intrinsic dimension.

row vector and column vector

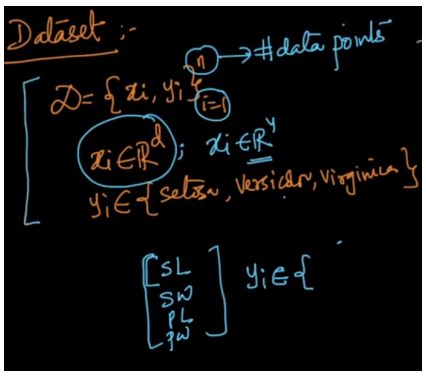
ith point is written as $x_i \in \mathbb{R}^d$

x_i here is a d-dimensional column vector (unless mentioned otherwise)
with components x_i1, x_i2, ..., x_id

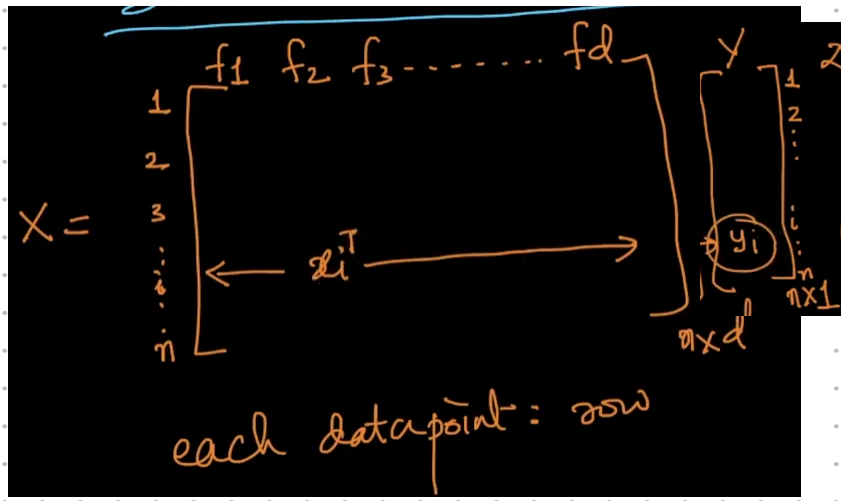
dataset:

a collection of data points (features) and corresponding outputs/ class labels

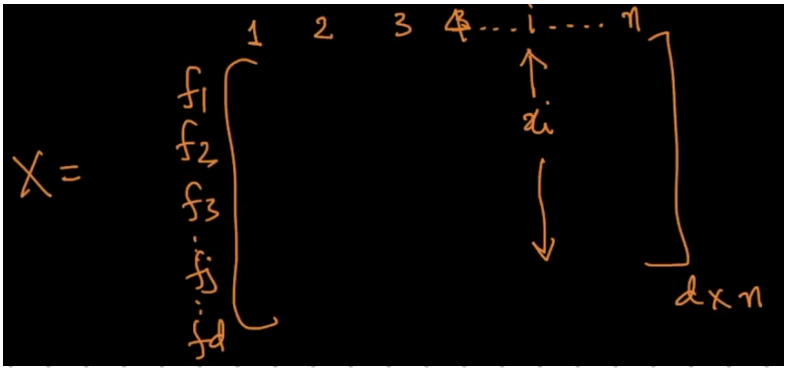
$D = \{X_i, y_i\}$
$X_i \in \mathbb{R}^d$



a dataset is also represented as a matrix



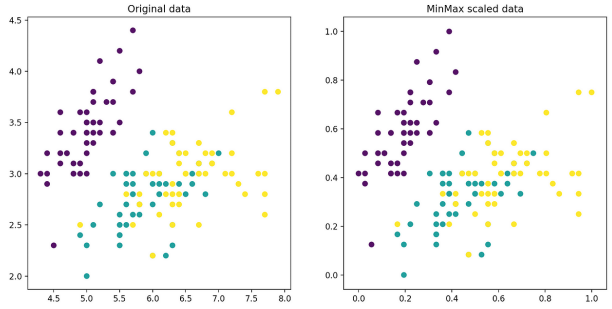
or



Data Preprocessing

min-max normalization also called feature scaling

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}$$



gets the values into the range [0, 1]
doesn't handle outliers very well

we do column normalization, we squish data to the range [0, 1]
if we collect heights and weights, heights in cm, weights in kg
and someone else collects in ft, lbs
then there are discrepancies between both data
through column normalization, we can handle either data as both lie between [0, 1]. we don't care about kgs, cms, or anything

we make the data scale independent

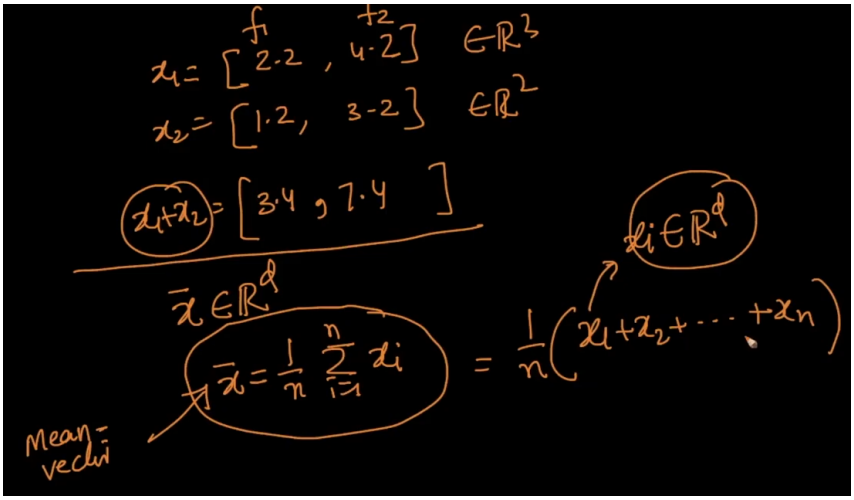
scale creates problems in various techniques

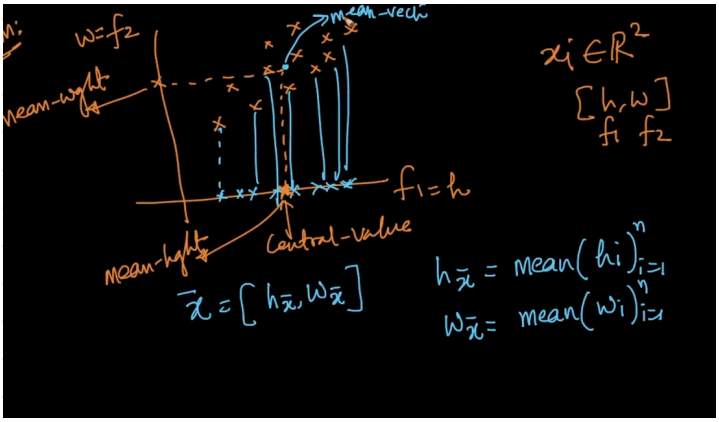
To rescale a range between an arbitrary set of values [a, b], the formula becomes:

$$x' = a + \frac{(x - \min(x))(b - a)}{\max(x) - \min(x)}$$

where a, b are the min-max values.

mean of data matrix:





basically component-wise mean

Standardization (Z-score normalization) mean centering + scaling

(type of mean normalization)

we convert data s.t. mean = 0, std dev = 1

$$x' = \frac{x - \bar{x}}{\max(x) - \min(x)}$$

Where x is the original feature vector, $\bar{x} = \text{average}(x)$ is the mean of that feature vector, and σ is its standard deviation.

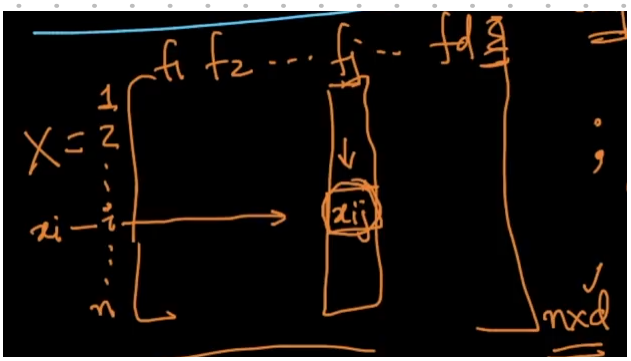
- **Min-max normalization**: Guarantees all features will have the exact same scale but does not handle outliers well.
- **Z-score normalization**: Handles outliers, but does not produce normalized data with the *exact* same scale.

Covariance Matrix

size dxd

S_ij = cov(x_i, x_j)

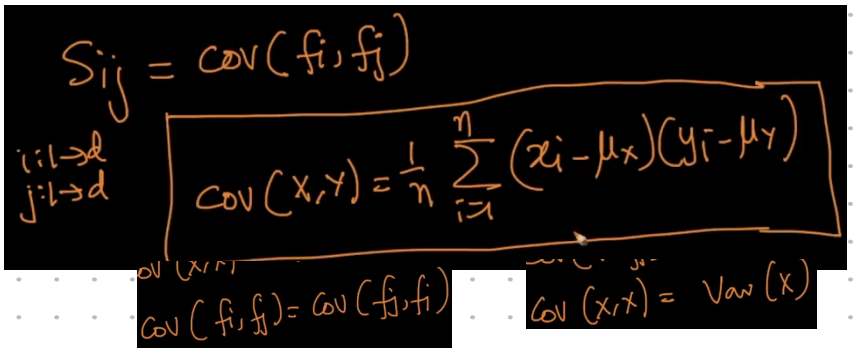
$$\operatorname{corr}(\mathbf{X}) = \begin{bmatrix} 1 & \frac{\mathrm{E}[(X_1-\mu_1)(X_2-\mu_2)]}{\sigma(X_1)\sigma(X_2)} & \cdots & \frac{\mathrm{E}[(X_1-\mu_1)(X_n-\mu_n)]}{\sigma(X_1)\sigma(X_n)} \\ \frac{\mathrm{E}[(X_2-\mu_2)(X_1-\mu_1)]}{\sigma(X_2)\sigma(X_1)} & 1 & \cdots & \frac{\mathrm{E}[(X_2-\mu_2)(X_n-\mu_n)]}{\sigma(X_2)\sigma(X_n)} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\mathrm{E}[(X_n-\mu_n)(X_1-\mu_1)]}{\sigma(X_n)\sigma(X_1)} & \frac{\mathrm{E}[(X_n-\mu_n)(X_2-\mu_2)]}{\sigma(X_n)\sigma(X_2)} & \cdots & 1 \end{bmatrix}.$$

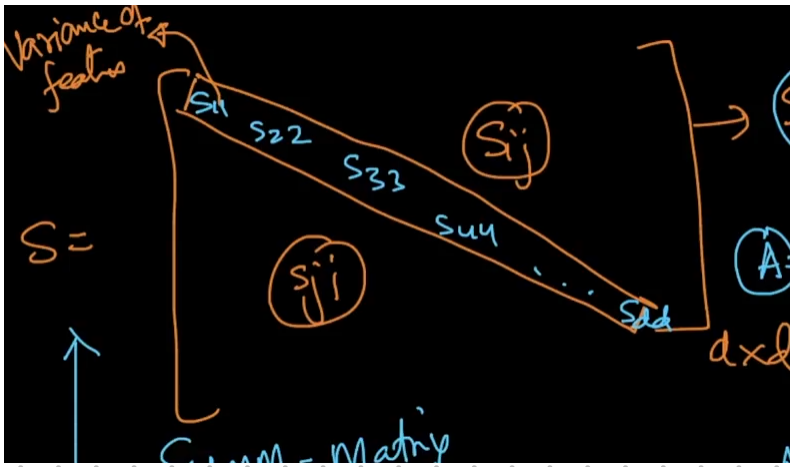


x_ij = jth feature for ith data point

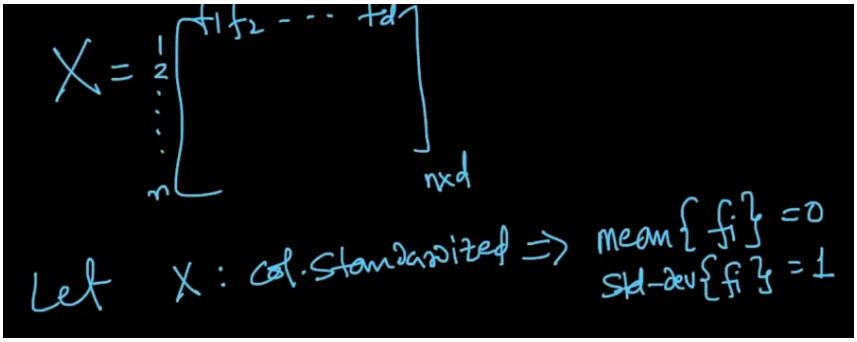
S_ij = element [i][j] in cov matrix

S_ij = cov(f_i, f_j)

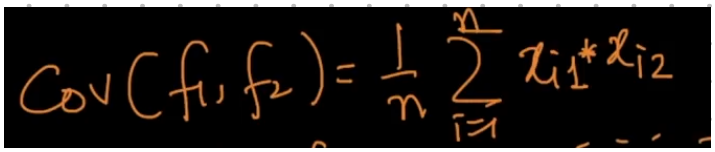




sq symmetric matrix, dxd



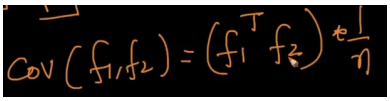
say we have data X
X is standardized,
so mean of columns (features) = 0,
std dev = 1



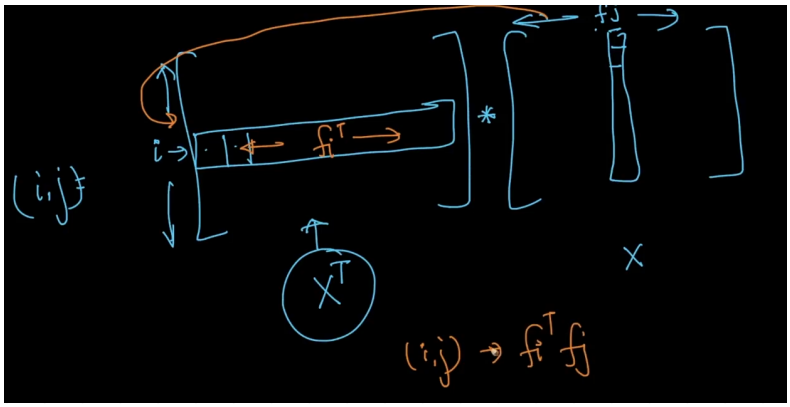
as mean = 0



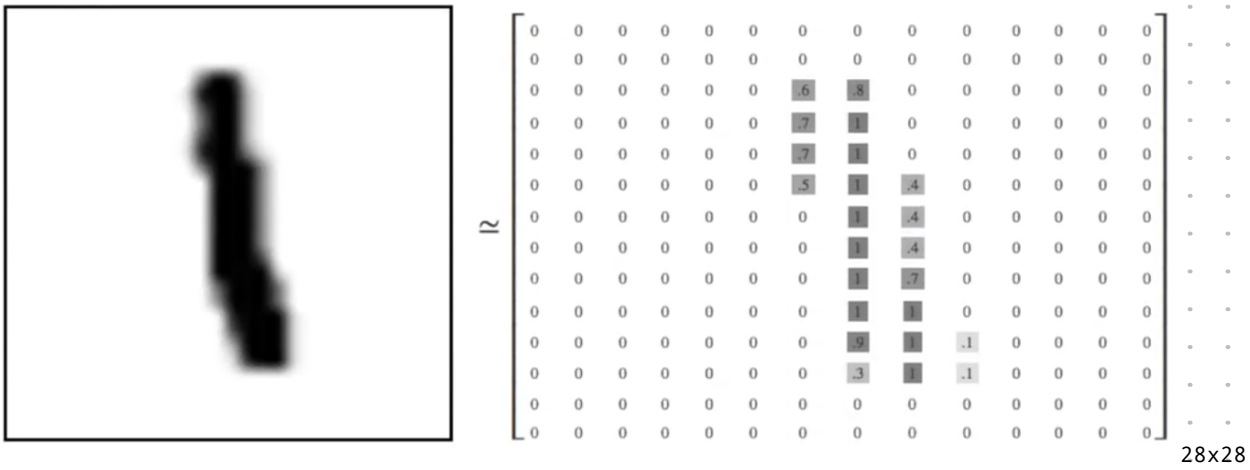
so we do element wise multiplication of f1, f2
we get dot product



so if data is normalized, $S_{d\times d} = \frac{1}{n}(X^T_{d\times n})(X_{n\times d})$

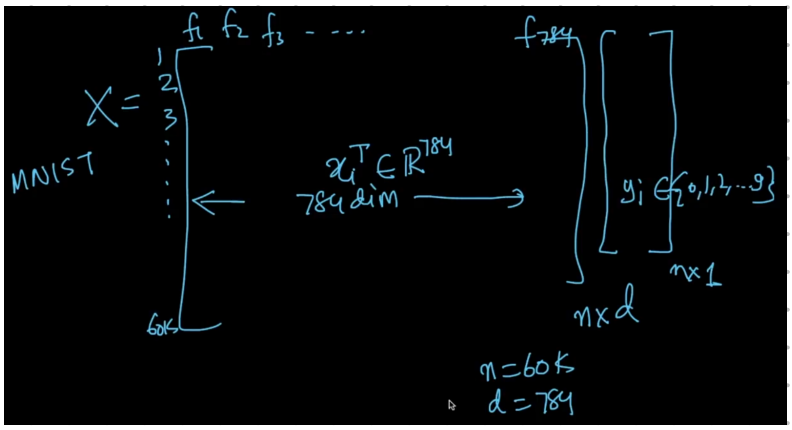


MNIST Dataset



we flatten the matrix by writing all the points sequentially in row major or column major.
we get a 784 x 1 vector from 28x28 matrix.

we get dataset with 784 features for one data point.



Loading MNIST

```
d0 = pd. read_csv("./mnist_train.csv")
d0.head()
```

	label	pixel0	pixel1	pixel2	pixel3	pixel4	pixel5	pixel
0	1	0	0	0	0	0	0	
1	0	0	0	0	0	0	0	
2	1	0	0	0	0	0	0	
3	4	0	0	0	0	0	0	
4	0	0	0	0	0	0	0	

5 rows × 785 columns

```
# we'll just separate the y (label) from X (features)
l = d0['label']
d = d0.drop('label', axis=1)
```

```
d.head()
```

	pixel0	pixel1	pixel2	pixel3	pixel4	pixel5	pixel6	pixel7	pixe
0	0	0	0	0	0	0	0	0	
1	0	0	0	0	0	0	0	0	
2	0	0	0	0	0	0	0	0	
3	0	0	0	0	0	0	0	0	
4	0	0	0	0	0	0	0	0	

5 rows × 784 columns

```
idx = 104
grid_data = d.iloc[idx].to_numpy().reshape(28, 28)
print(l[idx])
plt.imshow(grid_data, interpolation="none", cmap="gray")
plt.show()
```

✓ 0.4s

